

# Evaluation of Retrieval-Augmented Generation Pipelines

*A Comparative Analysis of Naive, HyDE, Hybrid, Reranking, and Multi-Query RAG Architectures*

Evaluation Framework: RAGAS + DeepEval (Reference-Based, No LLM Judge)  
Dataset: Brazilian E-Commerce (Olist) Knowledge Base — 100 Evaluation Queries  
Generated: 05 May 2026

| Pipeline       | Overall Score | Rank |
|----------------|---------------|------|
| Hybrid RAG     | 0.4633        | #1   |
| Reranking RAG  | 0.3655        | #2   |
| Naive RAG      | 0.3486        | #3   |
| HyDE RAG       | 0.3466        | #4   |
| MultiQuery RAG | 0.3457        | #5   |

Table 1: Overall performance ranking of all five RAG pipelines.

## 1. Introduction

---

Retrieval-Augmented Generation (RAG) is a technique that improves the accuracy of Large Language Models (LLMs) by first retrieving relevant documents from a knowledge base and then using those documents as context when generating an answer. Instead of relying solely on information memorised during training, the LLM reads the retrieved documents and grounds its response in real, up-to-date evidence. This makes RAG especially useful in domains where precise factual answers matter, such as e-commerce analytics.

This document evaluates five different RAG architectures applied to a Brazilian e-commerce knowledge base derived from the Olist dataset. Each pipeline was tested against the same set of 100 questions spanning three difficulty levels (easy, medium, and hard) and three question types (factual, analytical, and comparison). All answers were evaluated using two established frameworks — RAGAS and DeepEval — with reference-based metrics that require no additional LLM calls, making the evaluation fast, reproducible, and cost-free.

The five pipelines evaluated are: (1) Naive RAG, the simplest baseline that uses only semantic vector search; (2) HyDE RAG, which generates a hypothetical answer before searching; (3) Hybrid RAG, which combines keyword and semantic search through score fusion; (4) Reranking RAG, which adds a cross-encoder model to re-score retrieved documents; and (5) Multi-Query RAG, which expands the original question into multiple sub-queries before retrieving. Each pipeline is analysed in detail in the sections that follow.

## 2. Evaluation Methodology

---

All pipelines were evaluated using the same 100-question golden dataset, the same language model (Llama-3.3-70b-versatile via Groq API), and the same retrieval depth (top-5 documents passed to the LLM). Evaluation was performed using two frameworks.

### 2.1 RAGAS Metrics

RAGAS provides five metrics to assess the quality of a RAG pipeline from different angles:

- **Faithfulness** — Measures whether every sentence in the generated answer is directly supported by the retrieved documents. A score of 1.0 means the answer is fully grounded; 0.0 means it contains information not found in any retrieved document.
- **Answer Relevancy** — Measures how closely the generated answer addresses the original question, computed using TF-IDF cosine similarity between the answer and the question.
- **Context Precision** — Measures whether the most relevant document was retrieved and placed at the top of the ranking (Average Precision at k). High precision means the pipeline is good at prioritising the right document.
- **Context Recall** — Measures how much of the expected answer's content is present in the retrieved documents. High recall means the pipeline found all the information needed to answer the question.
- **Factual Correctness** — Compares the generated answer to the gold-standard answer using ROUGE-L F1, which measures token overlap. Consistently low across all pipelines because LLMs tend to generate verbose answers while gold answers are short and precise.

## 2.2 DeepEval Metrics

DeepEval provides six complementary metrics, all computed without an LLM judge:

- **Answer Relevancy** — Same computation as RAGAS Answer Relevancy (TF-IDF cosine similarity).
- **Faithfulness** — Same computation as RAGAS Faithfulness (sentence-level support check).
- **Contextual Precision** — Same computation as RAGAS Context Precision.
- **Contextual Recall** — Same computation as RAGAS Context Recall.
- **Contextual Relevancy** — Measures how relevant each individual retrieved document is to the question, averaged across all five documents. Low scores indicate that many retrieved documents are not directly useful for answering the question.
- **Hallucination** — Defined as 1.0 minus Faithfulness. A high hallucination score means the answer contains many statements not grounded in the retrieved documents.

### 3. Comparative Overview

The table below summarises the RAGAS mean scores for all five pipelines across 100 queries. Green cells indicate the best-performing pipeline for each metric. The overall score is the mean of all eleven RAGAS and DeepEval metrics combined.

| Pipeline       | Faithfulness | Ans.Relevancy | Ctx.Precision | Ctx.Recall | Factual Corr. | Overall |
|----------------|--------------|---------------|---------------|------------|---------------|---------|
| Hybrid RAG     | 0.4072       | 0.5897        | 0.5445        | 0.5637     | 0.1563        | 0.4633  |
| Reranking RAG  | 0.2432       | 0.5006        | 0.4095        | 0.3565     | 0.1215        | 0.3655  |
| Naive RAG      | 0.2679       | 0.5077        | 0.3225        | 0.3338     | 0.1161        | 0.3486  |
| HyDE RAG       | 0.2341       | 0.5152        | 0.3117        | 0.3437     | 0.1131        | 0.3466  |
| MultiQuery RAG | 0.2455       | 0.5109        | 0.3132        | 0.3341     | 0.1192        | 0.3457  |

Table 2: Mean RAGAS scores across 100 evaluation queries for all five RAG pipelines. Green cells indicate the highest score in each column.

At a high level, Hybrid RAG outperforms all other pipelines on every retrieval metric. Reranking RAG comes second, benefiting from its post-retrieval re-scoring step. Naive RAG, HyDE RAG, and Multi-Query RAG perform similarly overall, with differences of less than 0.003 in their total mean score, suggesting that query expansion and hypothetical document generation do not provide meaningful improvements over a simple semantic search baseline on this structured e-commerce dataset.



### 4.1 Overview

Hybrid RAG is the most sophisticated retrieval pipeline evaluated in this study. It retrieves documents using two independent methods simultaneously: BM25, a classical keyword-based ranking algorithm, and ChromaDB semantic vector search using the all-MiniLM-L6-v2 embedding model. Both methods independently retrieve their top-10 candidate documents. The two ranked lists are then merged using Reciprocal Rank Fusion (RRF) with a smoothing constant  $k=60$ , which assigns each document a fusion score based on its position in each list. The top-5 documents from the fused list are passed to the LLM for answer generation. This approach is designed to combine the precision of keyword matching with the flexibility of semantic understanding.

### 4.2 Performance Metrics

| Metric                        | Score         |
|-------------------------------|---------------|
| RAGAS Faithfulness            | <b>0.4072</b> |
| RAGAS Answer Relevancy        | <b>0.5897</b> |
| RAGAS Context Precision       | <b>0.5445</b> |
| RAGAS Context Recall          | <b>0.5637</b> |
| RAGAS Factual Correctness     | <b>0.1563</b> |
| DeepEval Contextual Relevancy | <b>0.1375</b> |
| DeepEval Hallucination        | <b>0.5928</b> |

Table: Mean evaluation scores for Hybrid RAG across 100 queries.

### 4.3 Key Observations

- Hybrid RAG achieved the highest scores across every retrieval metric in this evaluation. Its Context Precision of 0.5445 and Context Recall of 0.5637 are both significantly higher than any other pipeline, meaning it more consistently retrieves the correct document and places it near the top of the ranked list.
- The Faithfulness score of 0.4072 is the highest among all pipelines. This means that when Hybrid RAG retrieves the correct context, the LLM is more likely to ground its answer in that context rather than making up information.
- On easy factual queries — such as 'What is the total payment value for order X?' or 'What was the revenue for category Y?' — BM25 consistently retrieves the exact matching document at rank 1 because the query contains the document's unique identifier verbatim. Semantic search alone often fails these queries because embedding similarity is not sensitive to specific alphanumeric strings.
- On medium and hard analytical queries, the semantic search component contributes more value by identifying thematically related documents that BM25 would miss. The RRF fusion step ensures that documents appearing in both lists are promoted, reducing noise.

- Factual Correctness (0.1563) remains low, consistent with all other pipelines. This is not a failure of retrieval but a limitation of the metric: ROUGE-L penalises verbose LLM answers even when the factual content is correct, because the gold answers are typically short numeric strings.

#### 4.4 Advantages

- ✓ Best overall performance: Hybrid RAG ranked first on all 11 metrics across the full 100-query evaluation, demonstrating consistent superiority rather than excelling only in specific scenarios.
- ✓ Handles both exact-match and semantic queries in a single pipeline. BM25 excels at keyword-heavy queries such as those containing order IDs, seller IDs, or category names, while semantic search handles paraphrased or conceptual questions.
- ✓ Single LLM call per query: despite combining two retrieval methods, Hybrid RAG uses only one LLM call for generation — the same as Naive RAG — making it cost-competitive.
- ✓ BM25 indexing is fast, fully offline, and requires no GPU. The BM25 index is built once and stored as a pickle file, adding negligible overhead compared to ChromaDB.
- ✓ RRF fusion is a robust, parameter-light method. It does not require training and is resilient to score distribution differences between BM25 and semantic search.
- ✓ Scales well to larger knowledge bases because BM25 and vector search are both computationally efficient at retrieval time.

#### 4.5 Disadvantages

- ✗ Requires maintaining two separate index structures — a BM25 pickle file and a ChromaDB vector store — increasing storage requirements and adding a setup step compared to single-index pipelines.
- ✗ BM25 tokenisation is case-sensitive and language-dependent. Queries written differently from the document text (e.g., different word order, abbreviations) may not benefit from the keyword retrieval component.
- ✗ The RRF smoothing constant  $k=60$  is a fixed hyperparameter. While it performs well in this evaluation, a different value may be more optimal for other datasets or query distributions, requiring tuning.
- ✗ Hallucination score of 0.5928 is the lowest across pipelines, meaning the LLM's answers are more tightly constrained by retrieved context. While this is generally desirable, it also means that when retrieved documents do not contain the answer, the model is less willing to reason beyond them.

## 5.1 Overview

Reranking RAG extends the Naive RAG pipeline by adding a post-retrieval refinement step. It begins by retrieving an initial pool of 20 candidate documents from ChromaDB using semantic vector search. These 20 documents are then passed through a cross-encoder model — specifically cross-encoder/ms-marco-MiniLM-L-6-v2 — which scores each document against the original query for direct relevance. The cross-encoder is more accurate than a bi-encoder because it reads both the query and the document together in the same forward pass, capturing fine-grained relevance signals. The top-5 documents by cross-encoder score are passed to the LLM for answer generation.

## 5.2 Performance Metrics

| Metric                        | Score  |
|-------------------------------|--------|
| RAGAS Faithfulness            | 0.2432 |
| RAGAS Answer Relevancy        | 0.5006 |
| RAGAS Context Precision       | 0.4095 |
| RAGAS Context Recall          | 0.3565 |
| RAGAS Factual Correctness     | 0.1215 |
| DeepEval Contextual Relevancy | 0.1229 |
| DeepEval Hallucination        | 0.7568 |

Table: Mean evaluation scores for Reranking RAG across 100 queries.

## 5.3 Key Observations

- Reranking RAG ranked second overall with a mean score of 0.3655. Its most notable improvement over Naive RAG is in Context Precision (0.4095 vs 0.3225), confirming that the cross-encoder successfully promotes the most relevant document to rank 1 more consistently than bi-encoder semantic search alone.
- The pipeline performs best on easy factual questions where the query contains enough specific terms for the cross-encoder to confidently score one document significantly higher than the rest. In these cases, Context Precision reaches 1.0 frequently.
- On hard analytical and comparison questions, the benefit of reranking is reduced because answering these questions requires multiple supporting documents rather than one definitive source. Reranking optimises for single-document relevance and cannot improve multi-document coverage.
- Context Recall (0.3565) is only marginally better than Naive RAG (0.3338). This confirms that reranking does not improve the coverage of information retrieved — it only improves the ordering of what was already retrieved. If the correct document was not in the initial top-20, reranking cannot recover it.



- Hallucination score of 0.7568 is high, indicating the model frequently generates statements not supported by the retrieved context — a pattern shared with Naive, HyDE, and MultiQuery RAG on this dataset.

## 5.4 Advantages

- ✓ Context Precision improvement: the cross-encoder reliably places the most relevant document at or near the top of the final list, which helps the LLM find the answer quickly without being distracted by lower-ranked documents.
- ✓ No additional LLM call is required. The cross-encoder is a lightweight local model that runs on CPU without GPU requirements, adding only a small latency overhead.
- ✓ Plug-and-play design: Reranking can be added on top of any existing vector search pipeline without modifying the indexing step or changing the LLM prompt.
- ✓ The cross-encoder provides interpretable relevance scores for each document, which can be used to filter out documents below a threshold before sending them to the LLM.
- ✓ Generally robust to vocabulary mismatch because the cross-encoder reads the full text of each document jointly with the query, unlike bi-encoder embeddings that encode them independently.

## 5.5 Disadvantages

- ✗ Retrieval is bounded by the initial pool: if the correct document does not appear in the top-20 semantic search results, the reranker cannot recover it. The pipeline's recall ceiling is set by the upstream vector search step.
- ✗ Running a cross-encoder on 20 documents per query adds inference latency compared to Naive RAG. On large pools, this latency can become significant without GPU acceleration.
- ✗ The reranker model (ms-marco-MiniLM-L-6-v2) was trained on MS MARCO, a web search dataset. It may not generalise perfectly to the structured, tabular-style documents in this e-commerce knowledge base.
- ✗ Does not address keyword-match failures: if the initial semantic pool consistently misses exact-match queries (order IDs, numeric values), the reranker cannot compensate because none of the candidates in the pool will score highly.
- ✗ Context Recall (0.3565) is barely better than Naive RAG (0.3338), showing that reranking alone is insufficient to improve the completeness of the retrieved information.

## 6.1 Overview

Naive RAG is the simplest and most widely used RAG architecture. It encodes each query into a dense vector using the all-MiniLM-L6-v2 sentence embedding model and retrieves the top-5 most similar documents from ChromaDB using cosine distance. The retrieved documents are concatenated into a prompt and sent to the LLM for answer generation. There is no reranking, no query expansion, and no additional retrieval step. This pipeline serves as the primary baseline against which all other architectures are compared in this study.

## 6.2 Performance Metrics

| Metric                        | Score  |
|-------------------------------|--------|
| RAGAS Faithfulness            | 0.2679 |
| RAGAS Answer Relevancy        | 0.5077 |
| RAGAS Context Precision       | 0.3225 |
| RAGAS Context Recall          | 0.3338 |
| RAGAS Factual Correctness     | 0.1161 |
| DeepEval Contextual Relevancy | 0.1229 |
| DeepEval Hallucination        | 0.7321 |

Table: Mean evaluation scores for Naive RAG across 100 queries.

## 6.3 Key Observations

- Naive RAG achieved an overall mean score of 0.3486, placing it third among the five pipelines. Given its simplicity and the absence of any retrieval enhancement, this result establishes a strong and competitive baseline.
- The pipeline performs well on easy factual queries where the question contains enough descriptive language (category names, delivery statuses, state codes) that the embedding similarity to the correct document is naturally high.
- Performance drops significantly on queries that contain only alphanumeric identifiers (order IDs, seller IDs) because embedding models do not encode unique identifiers meaningfully. The cosine similarity between 'total payment for order d0eb69a...' and the corresponding order document is not necessarily higher than for unrelated order documents.
- Hallucination score of 0.7321 indicates that the LLM frequently generates statements beyond what is supported by the retrieved documents. This is particularly common on analytical and comparison questions where the retrieved documents contain relevant but insufficient information.
- Context Precision (0.3225) and Context Recall (0.3338) are both the lowest among the five pipelines for retrieval-focused metrics, confirming that single-stage semantic search is the weakest retrieval strategy evaluated.

## 6.4 Advantages

- ✓ Simplest architecture: the entire pipeline consists of one embedding model and one vector database. There are no additional components, models, or indexes to maintain.
- ✓ Lowest computational cost: a single embedding lookup and one LLM call per query. No GPU is required beyond what the LLM inference endpoint provides.
- ✓ Fastest setup time: building the ChromaDB collection from scratch takes a few minutes and requires no configuration beyond choosing the embedding model.
- ✓ Fully deterministic: given the same query and the same index, the pipeline always retrieves the same documents, making it easy to debug and reproduce.
- ✓ Serves as the most interpretable baseline: any score improvement in other pipelines can be directly attributed to the enhancement they add over this configuration.

## 6.5 Disadvantages

- ✗ Fails on exact-match queries: queries containing specific identifiers (order IDs, seller IDs, product category slugs) are not well handled by semantic embeddings, which do not distinguish between similar-length alphanumeric strings.
- ✗ No mechanism for quality control: the top-5 documents are always passed to the LLM regardless of their actual relevance. If all five documents are weakly related to the query, the LLM has no useful context and is more likely to hallucinate.
- ✗ Lowest Context Precision among all pipelines (0.3225), meaning the most relevant document is frequently not ranked first, causing the LLM to read less useful documents first and potentially anchor its answer on them.
- ✗ Cannot separate keyword relevance from semantic relevance — queries that benefit from both (the majority of real-world queries) are underserved by a single-channel approach.

### 7.1 Overview

HyDE (Hypothetical Document Embedding) RAG addresses a fundamental limitation of standard semantic search: a short question and a long answer document may have very different embeddings even when they are topically related, because their language and length differ significantly. HyDE mitigates this by first asking the LLM to generate a hypothetical answer to the query (LLM call 1). This hypothetical answer — which resembles a real knowledge-base document in structure and vocabulary — is then embedded and used as the retrieval query instead of the original question. The top-5 most similar documents are retrieved from ChromaDB, and the LLM generates the final answer using those documents as context (LLM call 2).

### 7.2 Performance Metrics

| Metric                        | Score  |
|-------------------------------|--------|
| RAGAS Faithfulness            | 0.2341 |
| RAGAS Answer Relevancy        | 0.5152 |
| RAGAS Context Precision       | 0.3117 |
| RAGAS Context Recall          | 0.3437 |
| RAGAS Factual Correctness     | 0.1131 |
| DeepEval Contextual Relevancy | 0.1244 |
| DeepEval Hallucination        | 0.7659 |

Table: Mean evaluation scores for HyDE RAG across 100 queries.

### 7.3 Key Observations

- HyDE RAG ranked fourth overall with a mean score of 0.3466, performing marginally worse than Naive RAG (0.3486) despite using twice as many LLM calls. This suggests that on structured, fact-dense e-commerce data, generating hypothetical answers introduces more noise than signal.
- The hypothetical documents generated by the LLM are plausible but factually invented. For example, when asked 'What is the total payment for order X?', the LLM might generate 'The total payment for order X is approximately 150 BRL', which embeds closer to generic order documents than to the specific order document containing the exact value. This misdirects retrieval.
- Faithfulness (0.2341) is the lowest of all five pipelines, meaning HyDE produces the least grounded answers. This is a direct consequence of hypothetical document hallucination cascading into poor retrieval, which then cascades into poor generation.
- HyDE performs relatively better on broad analytical questions where the hypothetical document captures the right thematic domain, improving semantic overlap with relevant documents. However, on this dataset, even these marginal gains are insufficient to outperform Naive RAG.

- Hallucination score of 0.7659 is the highest across all pipelines, confirming that HyDE's retrieval is the least reliable at returning the context needed for grounded answers.

## 7.4 Advantages

- ✓ Effective on vague or conceptual queries where the question is phrased very differently from the language used in knowledge-base documents. The hypothetical answer bridges the vocabulary gap.
- ✓ Requires no changes to the indexing pipeline. HyDE operates entirely at query time using the same ChromaDB collection as Naive RAG.
- ✓ Can improve retrieval for open-domain question-answering tasks where documents are narrative in style and the query is conversational — a scenario where the vocabulary gap between questions and answers is large.
- ✓ No additional model or index is required beyond the LLM and the vector store. The hypothetical document generation is an additional prompt, not an additional component.

## 7.5 Disadvantages

- ✗ Doubles the LLM API cost: every query requires one call for hypothetical document generation and one call for final answer generation, increasing token consumption and response latency significantly.
- ✗ Hypothetical documents frequently contain hallucinated facts. On a structured e-commerce knowledge base where documents contain precise numeric values and unique identifiers, any factual drift in the hypothetical document actively misleads retrieval.
- ✗ Worst faithfulness score (0.2341) of all five pipelines — the cascade from poor retrieval to poor generation is measurable and significant.
- ✗ Provides no benefit over Naive RAG on this dataset: Context Precision is lower (0.3117 vs 0.3225), Context Recall is slightly higher (0.3437 vs 0.3338), and overall mean score is lower (0.3466 vs 0.3486).
- ✗ Unsuitable for queries requiring exact-match retrieval — hypothetical answers cannot reproduce specific order IDs or category slugs accurately, making the embedding no more useful than the original question embedding for these cases.

### 8.1 Overview

Multi-Query RAG addresses the problem that a single question phrasing may not capture all relevant semantic angles of the information need. It uses the LLM to generate three additional rephrasings of the original query (LLM call 1), producing four sub-queries in total. Each sub-query independently retrieves its top-10 documents from ChromaDB. All retrieved documents across the four sub-query lists are then fused using Reciprocal Rank Fusion (RRF), and the top-5 documents from the fused list are passed to the LLM for final answer generation (LLM call 2). The intent is to improve recall by casting a wider semantic net over the knowledge base.

### 8.2 Performance Metrics

| Metric                        | Score  |
|-------------------------------|--------|
| RAGAS Faithfulness            | 0.2455 |
| RAGAS Answer Relevancy        | 0.5109 |
| RAGAS Context Precision       | 0.3132 |
| RAGAS Context Recall          | 0.3341 |
| RAGAS Factual Correctness     | 0.1192 |
| DeepEval Contextual Relevancy | 0.1213 |
| DeepEval Hallucination        | 0.7545 |

Table: Mean evaluation scores for Multi-Query RAG across 100 queries.

### 8.3 Key Observations

- Multi-Query RAG ranked last overall with a mean score of 0.3457, despite being one of the most computationally intensive pipelines. This outcome demonstrates that adding more retrieval complexity does not always translate into better performance, especially on structured, domain-specific datasets.
- Context Recall (0.3341) is essentially identical to Naive RAG (0.3338). This means that generating multiple sub-queries and retrieving up to 40 candidate documents provides no improvement in the coverage of information relevant to the expected answer.
- Context Precision (0.3132) is the lowest of all pipelines alongside HyDE (0.3117). The multi-query RRF fusion process boosts documents that appear frequently across sub-query lists. On this dataset, generic summary documents (e.g., top-level category summaries, delivery status summaries) appear consistently across all sub-query lists and therefore rank highly after fusion, even when they are not the most relevant document for the specific query.
- On easy factual queries, the LLM-generated sub-queries are semantically similar to the original query and all retrieve nearly identical document sets. RRF fusion in this scenario amplifies noise rather than diversity.

- Hallucination score of 0.7545 confirms that, like Naive and HyDE RAG, the pipeline frequently generates answers not grounded in the retrieved context.

## 8.4 Advantages

- ✓ Improved semantic coverage: when a query is ambiguous or can be interpreted in multiple ways, generating diverse sub-queries increases the chance that at least one of them retrieves the relevant document.
- ✓ Useful for open-domain knowledge bases with varied document styles. When documents use different vocabulary to describe the same concept, different sub-query phrasings can surface documents that a single query embedding would miss.
- ✓ RRF fusion across multiple sub-query lists is a robust aggregation method that reduces dependence on any single query phrasing, making the pipeline more robust to slight variations in how users ask questions.
- ✓ No changes to the indexing pipeline required: Multi-Query RAG operates at query time using the same ChromaDB collection as Naive RAG.

## 8.5 Disadvantages

- ✗ Highest operational cost: two LLM calls plus up to 40 ChromaDB similarity searches per query. This cost is significantly higher than Naive RAG with no measurable performance benefit on this dataset.
- ✗ LLM-generated sub-queries on factual questions tend to be near-identical paraphrases rather than genuinely diverse reformulations. For example, 'What is the total payment for order X?' generates variations like 'What was the payment amount for order X?' — which retrieve almost the same documents as the original query.
- ✗ RRF fusion of many similar retrieval lists amplifies generic, broadly relevant documents (which rank consistently across all lists) over specific, query-relevant documents (which may only rank highly in one list). This reduces Context Precision.
- ✗ Lowest overall score (0.3457) among all five pipelines despite being the most compute-intensive. This represents the worst cost-to-performance ratio in the evaluation.
- ✗ Not suited for exact-match queries: query expansion cannot generate a sub-query that matches a unique document identifier (order ID, seller ID) any better than the original query does, so the fundamental retrieval failure of semantic search on these queries is not addressed.

## 9. Conclusion

---

This study evaluated five Retrieval-Augmented Generation pipelines on a structured Brazilian e-commerce knowledge base using 100 questions of varying difficulty and type. The evaluation used eleven reference-based metrics from RAGAS and DeepEval, providing a comprehensive and reproducible assessment of each pipeline's retrieval and generation quality.

The central finding of this evaluation is that retrieval quality is the primary determinant of RAG performance. Hybrid RAG, which combines BM25 keyword retrieval and semantic vector search through Reciprocal Rank Fusion, achieved the highest scores on every retrieval metric by a significant margin (Context Precision: 0.5445, Context Recall: 0.5637, Faithfulness: 0.4072). This demonstrates that different query types benefit from different retrieval mechanisms, and combining both is more effective than using either alone.

Reranking RAG finished second (0.3655), confirming that post-retrieval re-scoring with a cross-encoder improves precision without requiring an entirely new retrieval strategy. However, its recall limitation — bounded by the initial top-20 semantic pool — prevents it from matching the coverage of Hybrid RAG.

Naive RAG (0.3486), HyDE RAG (0.3466), and Multi-Query RAG (0.3457) performed at similar levels, with differences smaller than 0.003 in overall mean score. This finding is instructive: both HyDE and Multi-Query RAG add significant computational cost (doubling LLM calls) and architectural complexity but fail to improve over the simple Naive baseline on this dataset. On a structured knowledge base where documents contain precise numeric values and unique identifiers, hypothetical document generation introduces retrieval noise, and query expansion produces near-duplicate sub-queries rather than genuinely diverse reformulations.

These results suggest that for structured, fact-dense domains such as e-commerce analytics, the priority should be improving the retrieval mechanism itself — specifically by combining keyword and semantic search — rather than elaborating the query or adding intermediate generation steps. Hybrid RAG achieves the best performance with the same single LLM call per query as Naive RAG, making it the most cost-effective advanced pipeline among those evaluated.

**Recommendation:** For production deployment on structured e-commerce or similar fact-dense knowledge bases, Hybrid RAG (BM25 + Semantic + RRF) is the recommended architecture. If maintaining two indexes is operationally impractical, Reranking RAG is the best single-index alternative. HyDE RAG and Multi-Query RAG are not recommended for this class of domain without dataset-specific tuning.